

University Degree in Data Science and Engineering  
2022-2026

# Parallel Programming of FEM in Julia

*Bachelor Thesis*

**Sergio Madrid Cuevas**

Tutors:

Sergio Llorente Romano

Luis Emilio García Castillo

Madrid, September 29, 2026



# Outline

- 1 Introduction & Objectives
- 2 Theoretical Background
- 3 Implementation
- 4 Verification
- 5 Conclusions & Future Work

# Outline

- 1 Introduction & Objectives
- 2 Theoretical Background
- 3 Implementation
- 4 Verification
- 5 Conclusions & Future Work

## The “Two-Language Problem”

- **Prototyping:** Easy to use but slow (Python, MATLAB).
- **Deploying:** Fast but complex to maintain (Fortran, C++).

*Result:* Algorithms are often written twice, slowing research.

## Motivation

- GREMA relies on HOFEM (in-house Fortran solver).
- **Julia as the Solution:** Combines Python-like interactivity with Fortran-like speed.

**Main Goal:** Develop a high-performance, fully parallelized Finite Element Method (FEM) assembly framework natively in Julia.

- **End-to-End Julia Pipeline**
- **MPI Parallelization**
- **Mathematical Verification**
- **HOFEM Interoperability**

# Outline

- 1 Introduction & Objectives
- 2 Theoretical Background**
- 3 Implementation
- 4 Verification
- 5 Conclusions & Future Work

# Strong Formulation

We consider the double-curl formulation of Maxwell's equations for the electric phasor:

$$\nabla \times (\boldsymbol{\mu}_r^{-1}(\nabla \times \mathbf{E})) - k_0^2 \epsilon_r \mathbf{E} = -ik_0 \eta_0 \mathbf{J} - \nabla \times (\boldsymbol{\mu}_r^{-1} \mathbf{M})$$

To ensure a unique solution, the following boundary conditions are enforced on  $\Gamma = \partial\Omega$ :

$$\begin{aligned} \mathbf{n} \times \mathbf{E} &= \mathbf{g}_D & \text{on } \Gamma_D \\ \mathbf{n} \times (\boldsymbol{\mu}_r^{-1}(\nabla \times \mathbf{E})) &= \mathbf{g}_N & \text{on } \Gamma_N \\ \mathbf{n} \times (\boldsymbol{\mu}_r^{-1}(\nabla \times \mathbf{E})) - ik_0 \eta_0 \mathbf{n} \times (\mathbf{n} \times \mathbf{E}) &= \mathbf{g}_C & \text{on } \Gamma_C \end{aligned}$$

We only consider Dirichlet boundary conditions.

# Weak Formulation

Find  $\mathbf{E} \in \mathbf{V}(\Omega)$  such that  $\forall \mathbf{v} \in \mathbf{V}(\Omega)$ :

$$\int_{\Omega} (\nabla \times \bar{\mathbf{v}}) \cdot (\mu_r^{-1} (\nabla \times \mathbf{E})) \, dV - k_0^2 \int_{\Omega} \bar{\mathbf{v}} \cdot (\epsilon_r \mathbf{E}) \, dV = -ik_0 \eta_0 \int_{\Omega} \bar{\mathbf{v}} \cdot \mathbf{J} \, dV - \int_{\Omega} \bar{\mathbf{v}} \cdot [\nabla \times (\mu_r^{-1} \mathbf{M})] \, dV$$

The space  $\mathbf{H}(\text{curl}, \Omega)$  ensures square-integrability of fields and their curls:

$$\mathbf{H}(\text{curl}, \Omega) = \{\mathbf{v} \in \mathbf{L}^2(\Omega) \mid \nabla \times \mathbf{v} \in \mathbf{L}^2(\Omega)\}$$

For homogeneous Dirichlet conditions ( $\mathbf{g}_D = 0$ ), the trial/test space is defined as:

$$\mathbf{V}(\Omega) = \{\mathbf{v} \in \mathbf{H}(\text{curl}, \Omega) \mid \mathbf{n} \times \mathbf{v} = 0 \text{ on } \Gamma_D\}$$

# The Reference Element

The domain  $\Omega$  is discretized into tetrahedral elements  $K$ , which are mapped to a reference element  $\hat{K}$  to simplify basis evaluation.

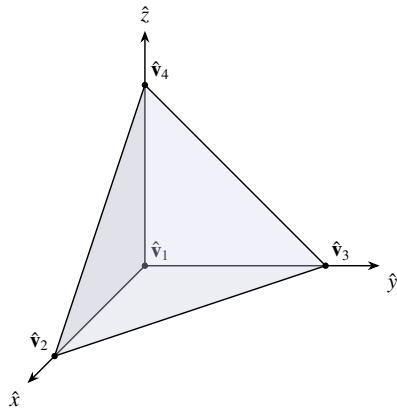
$\hat{K}$  is the right unit tetrahedron, given by vertices:

$$\hat{\mathbf{v}}_1 = (0, 0, 0)^\top, \quad \hat{\mathbf{v}}_2 = (1, 0, 0)^\top, \quad \hat{\mathbf{v}}_3 = (0, 1, 0)^\top, \quad \hat{\mathbf{v}}_4 = (0, 0, 1)^\top$$

Affine mapping  $\Phi_K : \hat{K} \rightarrow K$  via barycentric coordinates  $\lambda_i(\hat{\mathbf{x}})$ :

$$\Phi_K(\hat{\mathbf{x}}) = \sum_{i=1}^4 \lambda_i(\hat{\mathbf{x}}) \mathbf{v}_i = \mathbf{v}_1 + \mathbf{J}_K^\top \hat{\mathbf{x}}$$

$$\text{with } \mathbf{J}_K = [\mathbf{v}_2 - \mathbf{v}_1 \quad \mathbf{v}_3 - \mathbf{v}_1 \quad \mathbf{v}_4 - \mathbf{v}_1]^\top$$



The reference tetrahedron  $\hat{K}$

Covariant Piola transformation for  $\mathbf{H}(\text{curl}, \Omega)$ :

$$\mathbf{E}|_K(\mathbf{x}) = \mathbf{J}_K^{-1} \hat{\mathbf{E}}(\hat{\mathbf{x}})$$

Contravariant Piola transformation for  $\mathbf{H}(\text{div}, \Omega)$ :

$$\nabla \times \mathbf{E}|_K(\mathbf{x}) = \frac{1}{\det(\mathbf{J}_K)} \mathbf{J}_K^\top (\hat{\nabla} \times \hat{\mathbf{E}})(\hat{\mathbf{x}})$$

# Finite Element Space

We choose  $(\hat{K}, \mathcal{N}_2 = (\mathbb{P}_1)^3 \oplus \mathbf{S}_2, \Sigma)$ . DoFs in  $\Sigma$  are chosen to match across interfaces:

- 12 edge DoFs (2 per edge):

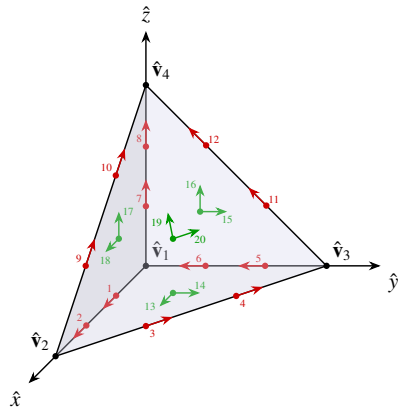
$$\ell_{\hat{e},q}(\mathbf{p}) = \int_{\hat{e}} (\mathbf{p} \cdot \mathbf{t}_{\hat{e}}) q \, dl, \quad \forall q \in \mathbb{P}_1(\hat{e})$$

- 8 face DoFs (2 per face):

$$\ell_{\hat{f},q}(\mathbf{p}) = \int_{\hat{f}} (\mathbf{p} \times \mathbf{n}_{\hat{f}}) \cdot \mathbf{q} \, dS, \quad \forall \mathbf{q} \in (\mathbb{P}_0)^2(\hat{f})$$

- Volume DoFs: not present

Elements sharing a face share DoFs which evaluate to the same value, ensuring conformity in  $\mathbf{H}(\text{curl}, \Omega)$ .



DoF placement on  $\hat{K}$

# Discretization

The continuous domain is approximated by a tetrahedral mesh  $\Omega \approx \mathcal{T}_h = \bigcup_{i=1}^N K_i$ .

We restrict the infinite-dimensional space  $\mathbf{V}(\Omega)$  to a finite-dimensional subspace  $\mathbf{V}_h(\Omega)$  defined on the mesh:

$$\mathbf{V}_h(\Omega) = \{\mathbf{v} \in \mathbf{V}(\Omega) \mid \mathbf{v}|_K \in \mathcal{N}_2(K), \forall K \in \mathcal{T}_h\}$$

Choosing a basis  $\{\mathbf{N}_i\}_{i=1}^N$  for  $\mathbf{V}_h(\Omega)$ , the electric field is approximated by:

$$\mathbf{E} \approx \mathbf{E}_h = \sum_{i=1}^N g_i \mathbf{N}_i$$

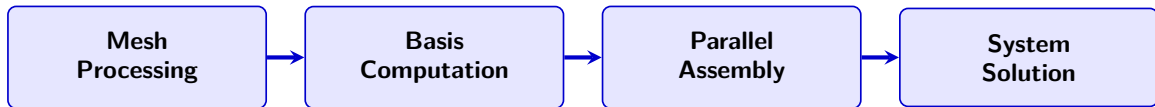
Testing with each basis function  $\mathbf{N}_j$ , the weak problem reduces to a linear system:

$$(\mathbf{K} - k_0^2 \mathbf{M}) \mathbf{g} = -(\mathrm{i} k_0 \eta_0 \mathbf{j} + \mathbf{m})$$

# Outline

- 1 Introduction & Objectives
- 2 Theoretical Background
- 3 Implementation**
- 4 Verification
- 5 Conclusions & Future Work

# Computational Workflow



$$\int_e g \, dl \approx \|\mathbf{v}_j - \mathbf{v}_i\|_2 \sum_{i=1}^n w'_i g(\Phi_e(x'_i))$$

$$\int_f g \, dS \approx \|(\mathbf{v}_j - \mathbf{v}_i) \times (\mathbf{v}_k - \mathbf{v}_i)\|_2 \sum_{i=1}^n \sum_{j=1}^n w'_i w'_j (1 - x'_i) g(\Phi_f(x'_i, x'_j (1 - x'_i)))$$

$$\int_K g \, dV \approx \det(\mathbf{J}_K) \sum_{i=1}^n \sum_{j=1}^n \sum_{k=1}^n w'_i w'_j w'_k (1 - x'_i)^2 (1 - x'_j) g(\Phi_K(x'_i, x'_j (1 - x'_i), x'_k (1 - x'_i) (1 - x'_j)))$$

where  $\Phi_e(\hat{x}) = (1 - \hat{x}) \mathbf{v}_i + \hat{x} \mathbf{v}_j$  and  $\Phi_f(\hat{x}, \hat{y}) = \mathbf{v}_i + \hat{x} (\mathbf{v}_j - \mathbf{v}_i) + \hat{y} (\mathbf{v}_k - \mathbf{v}_i)$ .

# Basis Computation

$$\mathbf{p} = \begin{bmatrix} a_1 + a_2x + a_3y + a_4z + Dy^2 - Fxy - Gxz + Hz^2 + Jyz \\ b_1 + b_2x + b_3y + b_4z - Dxy - Eyz + Fx^2 + Iz^2 - Jxz + Kxz \\ c_1 + c_2x + c_3y + c_4z + Ey^2 + Gx^2 - Hxz - Iyz - Kxy \end{bmatrix}$$



$$\begin{array}{llll} a_1 : \mathbf{m}_1 = (1, 0, 0)^\top, & a_2 : \mathbf{m}_2 = (x, 0, 0)^\top, & a_3 : \mathbf{m}_3 = (y, 0, 0)^\top, & a_4 : \mathbf{m}_4 = (z, 0, 0)^\top, \\ b_1 : \mathbf{m}_5 = (0, 1, 0)^\top, & b_2 : \mathbf{m}_6 = (0, x, 0)^\top, & b_3 : \mathbf{m}_7 = (0, y, 0)^\top, & b_4 : \mathbf{m}_8 = (0, z, 0)^\top, \\ c_1 : \mathbf{m}_9 = (0, 0, 1)^\top, & c_2 : \mathbf{m}_{10} = (0, 0, x)^\top, & c_3 : \mathbf{m}_{11} = (0, 0, y)^\top, & c_4 : \mathbf{m}_{12} = (0, 0, z)^\top, \\ D : \mathbf{m}_{13} = (y^2, -xy, 0)^\top, & E : \mathbf{m}_{14} = (0, -yz, y^2)^\top, & F : \mathbf{m}_{15} = (-xy, x^2, 0)^\top, & G : \mathbf{m}_{16} = (-xz, 0, x^2)^\top, \\ H : \mathbf{m}_{17} = (z^2, 0, -xz)^\top, & I : \mathbf{m}_{18} = (0, z^2, -yz)^\top, & J : \mathbf{m}_{19} = (yz, -xz, 0)^\top, & K : \mathbf{m}_{20} = (0, xz, -xy)^\top, \end{array}$$



$$\begin{bmatrix} (a_1)_1 & (a_1)_2 & \cdots & (a_1)_{20} \\ (a_2)_1 & (a_2)_2 & \cdots & (a_2)_{20} \\ \vdots & \vdots & \ddots & \vdots \\ J_1 & J_2 & \cdots & J_{20} \\ K_1 & K_2 & \cdots & K_{20} \end{bmatrix} = \begin{bmatrix} \ell_1(\mathbf{m}_1) & \ell_1(\mathbf{m}_2) & \cdots & \ell_1(\mathbf{m}_{20}) \\ \ell_2(\mathbf{m}_1) & \ell_2(\mathbf{m}_2) & \cdots & \ell_2(\mathbf{m}_{20}) \\ \vdots & \vdots & \ddots & \vdots \\ \ell_{19}(\mathbf{m}_1) & \ell_{19}(\mathbf{m}_2) & \cdots & \ell_{19}(\mathbf{m}_{20}) \\ \ell_{20}(\mathbf{m}_1) & \ell_{20}(\mathbf{m}_2) & \cdots & \ell_{20}(\mathbf{m}_{20}) \end{bmatrix}^{-1}$$

Integrals are evaluated on the reference element  $\hat{K}$  using Piola transformations:

$$\begin{aligned}(\mathbf{K}_K)_{i,j} &= \int_{\hat{K}} (\hat{\nabla} \times \hat{\mathbf{N}}_j)^\top \left[ \frac{\mathbf{J}_K \boldsymbol{\mu}_r^{-1} \mathbf{J}_K^\top}{\det(\mathbf{J}_K)} \right] (\hat{\nabla} \times \hat{\mathbf{N}}_i) d\hat{V} & (\mathbf{j}_K)_j &= \int_{\hat{K}} \hat{\mathbf{N}}_j^\top \left[ \mathbf{J}_K^{-\top} \det(\mathbf{J}_K) \right] (\mathbf{J} \circ \boldsymbol{\Phi}_K) d\hat{V} \\(\mathbf{M}_K)_{i,j} &= \int_{\hat{K}} \hat{\mathbf{N}}_j^\top \left[ \mathbf{J}_K^{-\top} \det(\mathbf{J}_K) \boldsymbol{\varepsilon}_r \mathbf{J}_K^{-1} \right] \hat{\mathbf{N}}_i d\hat{V} & (\mathbf{m}_K)_j &= \int_{\hat{K}} \hat{\mathbf{N}}_j^\top \left[ \mathbf{J}_K^{-\top} \det(\mathbf{J}_K) \right] (\nabla \times (\boldsymbol{\mu}_r^{-1} \mathbf{M}) \circ \boldsymbol{\Phi}_K) d\hat{V}\end{aligned}$$

Local DoFs are mapped to global DoFs using a mapping matrix  $\mathbf{B}$ .

# Parallel Boundary Condition Enforcement

We evaluate the DoF functionals directly on the boundary field  $\mathbf{g}_D$ :

$$g_I = \ell_I(\mathbf{g}_D) \quad \forall I \in \Gamma_D$$

The Nédélec functionals filter out normal components.

For local DoFs  $i, j$  in each element, mapping to global DoFs  $I, J$ :

- **If**  $I \in \Gamma_D$ : Skip local matrix assembly for this row.
- **If**  $J \in \Gamma_D$ : Move the known boundary contribution to the local RHS:

$$\mathbf{b}_I \leftarrow \mathbf{b}_I - \mathbf{A}_{I,J} g_J, \quad \mathbf{A}_{I,J} \leftarrow 0$$

- **Otherwise**: Assemble  $\mathbf{A}_{I,J}$  and  $\mathbf{b}_I$  normally.
- **Post-Assembly**: Globally set the diagonals  $\mathbf{A}_{I,I} = 1$  and  $\mathbf{b}_I = g_I$  for all  $I \in \Gamma_D$ .

# Parallel Boundary Condition Enforcement

$$\begin{bmatrix} \mathbf{A}_{1,1} & \mathbf{A}_{1,2} & \mathbf{A}_{1,3} \\ \mathbf{A}_{2,1} & \mathbf{A}_{2,2} & \mathbf{A}_{2,3} \\ \mathbf{A}_{3,1} & \mathbf{A}_{3,2} & \mathbf{A}_{3,3} \end{bmatrix} \begin{bmatrix} \mathbf{x}_1 \\ \mathbf{x}_2 \\ \mathbf{x}_3 \end{bmatrix} = \begin{bmatrix} \mathbf{b}_1 \\ \mathbf{b}_2 \\ \mathbf{b}_3 \end{bmatrix}$$

↓

$$\begin{bmatrix} \mathbf{A}_{1,1} & 0 & \mathbf{A}_{1,3} \\ 0 & 1 & 0 \\ \mathbf{A}_{3,1} & 0 & \mathbf{A}_{3,3} \end{bmatrix} \begin{bmatrix} \mathbf{x}_1 \\ \mathbf{x}_2 \\ \mathbf{x}_3 \end{bmatrix} = \begin{bmatrix} \mathbf{b}_1 - \mathbf{A}_{1,2}\mathbf{g} \\ \mathbf{g} \\ \mathbf{b}_3 - \mathbf{A}_{3,2}\mathbf{g} \end{bmatrix}$$

# Outline

- 1 Introduction & Objectives
- 2 Theoretical Background
- 3 Implementation
- 4 Verification**
- 5 Conclusions & Future Work

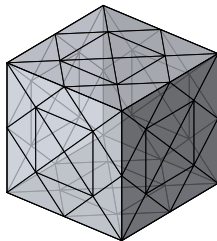
# Test Meshes

## Mesh 1:

192 elements

1468 DoFs

$h \approx 0.6124$

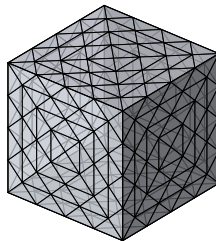


## Mesh 2:

1536 elements

10 712 DoFs

$h \approx 0.3062$

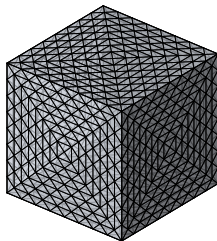


## Mesh 3:

12 288 elements

81 712 DoFs

$h \approx 0.1531$

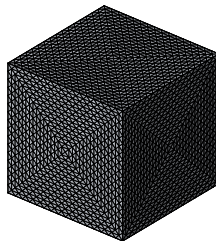


## Mesh 4:

98 304 elements

638 048 DoFs

$h \approx 0.0765$



## Mass Matrix (**M**)

### Properties:

- Symmetric
- Positive Definite

### Validation:

Passes for all 4 meshes.

## Stiffness Matrix (**K**)

### Properties:

- Symmetric
- Positive Semi-Definite
- Zero eigenvalues:  
 $N_{\text{vert}} + N_{\text{edge}} - 1$

### Validation:

Passes for first 2 meshes.

Zero eigenvalues: 364 and 2456.

# Method of Manufactured Solutions (MMS)

Used for verifying the correctness of numerical implementations.

- 1 Choose an analytical solution  $\mathbf{V}_{\text{MMS}}$ .
- 2 Substitute  $\mathbf{V}_{\text{MMS}}$  into the strong form to obtain  $\mathbf{J}_{\text{MMS}}$ .
- 3 Solve the FEM problem using  $\mathbf{J}_{\text{MMS}}$ . Compare the approximation  $\mathbf{E}_h$  against  $\mathbf{V}_{\text{MMS}}$ .

Three equations tested:

- **Test 1:**  $\mathbf{E} = \mathbf{V}_{\text{MMS}}$ ,  $e = e_f$ .
- **Test 2:**  $\nabla \times \mathbf{E} = \nabla \times \mathbf{V}_{\text{MMS}}$ ,  $e = e_c$ .
- **Test 3:**  $\nabla \times (\nabla \times \mathbf{E}) - k_0^2 \mathbf{E} = \nabla \times (\nabla \times \mathbf{V}_{\text{MMS}}) - k_0^2 \mathbf{V}_{\text{MMS}}$ ,  $e = e_f + e_c$ .

The errors are defined as:

$$e_f(\mathbf{V}_{\text{MMS}}, \mathbf{E}_h) = \frac{\sqrt{\int_{\Omega} \|\mathbf{V}_{\text{MMS}} - \mathbf{E}_h\|_2^2 dV}}{\sqrt{\int_{\Omega} \|\mathbf{V}_{\text{MMS}}\|_2^2 dV}} \quad e_c(\mathbf{V}_{\text{MMS}}, \mathbf{E}_h) = \frac{\sqrt{\int_{\Omega} \|\nabla \times \mathbf{V}_{\text{MMS}} - \nabla \times \mathbf{E}_h\|_2^2 dV}}{\sqrt{\int_{\Omega} \|\nabla \times \mathbf{V}_{\text{MMS}}\|_2^2 dV}}$$

MMS:  $\mathbf{V}_{\text{MMS}} \in \mathbf{V}_h(\Omega)$

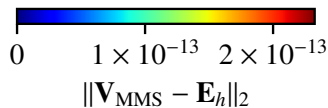
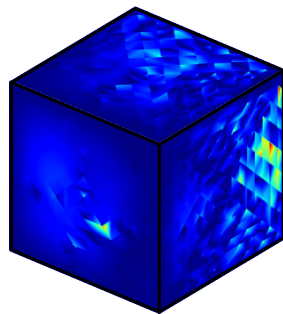
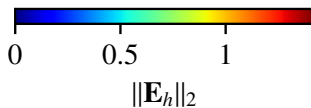
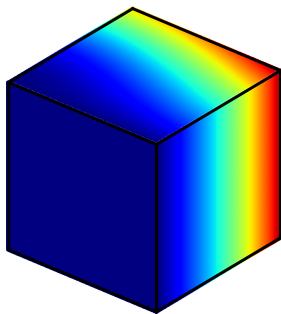
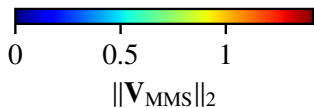
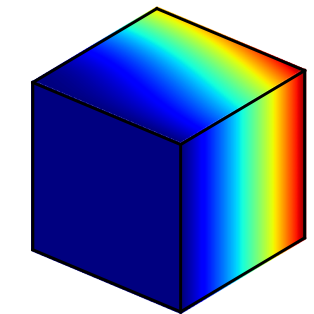
We first test a polynomial solution in the Nédélec finite element space:

$$\mathbf{V}_{\text{MMS}}(\mathbf{x}) = \begin{bmatrix} y^2 \\ -xy \\ 0 \end{bmatrix}$$

Since  $\mathbf{V}_{\text{MMS}}$  can be represented exactly by the basis functions, the discretization error is bounded by floating-point arithmetic.

|               | Test 1                   | Test 2                   | Test 3                   |
|---------------|--------------------------|--------------------------|--------------------------|
| <b>Mesh 1</b> | $1.4891 \times 10^{-15}$ | $1.6140 \times 10^{-13}$ | $2.9665 \times 10^{-13}$ |
| <b>Mesh 2</b> | $1.5508 \times 10^{-15}$ | $3.1853 \times 10^{-13}$ | $3.9999 \times 10^{-13}$ |
| <b>Mesh 3</b> | $1.5853 \times 10^{-15}$ | $6.4789 \times 10^{-12}$ | $2.0983 \times 10^{-13}$ |

MMS:  $\mathbf{V}_{\text{MMS}} \in \mathbf{V}_h(\Omega)$



# MMS: $\mathbf{V}_{\text{MMS}} \notin \mathbf{V}_h(\Omega)$

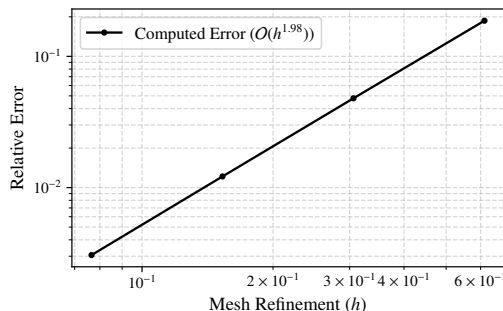
We now test a smooth solution outside the finite element space

$$\mathbf{V}_{\text{MMS}}(\mathbf{x}) = \begin{bmatrix} (\cos(\theta) \cos(\phi) - \sin(\phi)) e^{-ik_0(\mathbf{k} \cdot \mathbf{x})} \\ (\cos(\theta) \sin(\phi) + \cos(\phi)) e^{-ik_0(\mathbf{k} \cdot \mathbf{x})} \\ -\sin(\theta) e^{-ik_0(\mathbf{k} \cdot \mathbf{x})} \end{bmatrix}$$

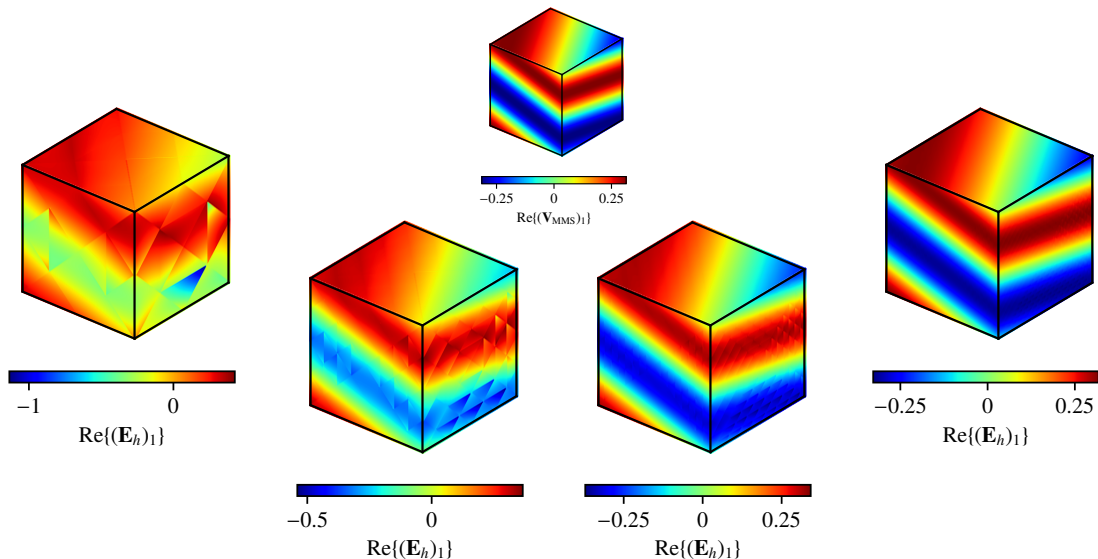
$$\mathbf{k} = [-\sin(\theta) \cos(\phi) \quad -\sin(\theta) \sin(\phi) \quad -\cos(\theta)]^T, \\ \theta = 20^\circ, \phi = 30^\circ, \text{ and } k_0 = 2\pi.$$

Because  $\mathbf{V}_{\text{MMS}} \notin \mathbf{V}_h(\Omega)$ , a discretization error is introduced. As  $h \rightarrow 0$ , this error decreases with rate  $\mathcal{O}(h^2)$ .

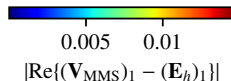
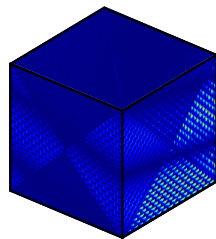
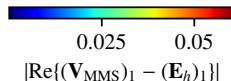
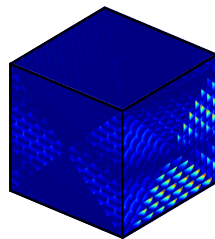
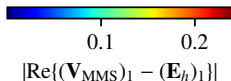
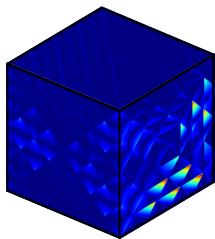
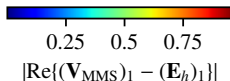
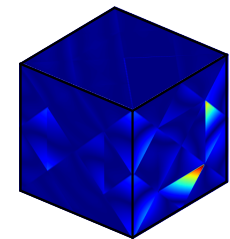
|               | Test 1   | Test 2   | Test 3   |
|---------------|----------|----------|----------|
| <b>Mesh 1</b> | 0.070132 | 0.094966 | 0.186892 |
| <b>Mesh 2</b> | 0.017448 | 0.024917 | 0.047917 |
| <b>Mesh 3</b> | 0.004287 | 0.006315 | 0.012162 |
| <b>Mesh 4</b> | 0.001059 | 0.001585 | 0.003062 |
| <b>Slope</b>  | 2.02     | 1.97     | 1.98     |



MMS:  $\mathbf{V}_{\text{MMS}} \notin \mathbf{V}_h(\Omega)$



MMS:  $\mathbf{V}_{\text{MMS}} \notin \mathbf{V}_h(\Omega)$



## Test 1: Step-by-Step Execution

- Run HOFEM step-by-step through the generated Julia interface.
- Verified on two different projects, “pyramid” and “one tet”.
- The interface-run code yielded the same results as the full HOFEM.

## Test 2: Matrix Injection

- Skip the Fortran assembly steps and instead pass the Julia-generated system directly to HOFEM to be solved.
- Not tested due to last minute issues.

# Outline

- 1 Introduction & Objectives
- 2 Theoretical Background
- 3 Implementation
- 4 Verification
- 5 Conclusions & Future Work**

- **Julia FEM Framework:**
  - Mesh processing.
  - Basis function computation.
  - Parallel system assembly.
- **Verification:** Matrix properties and  $\mathcal{O}(h^2)$  convergence.
- **HOFEM Interoperability:** Executed HOFEM step-by-step through the Julia interface.

- **Boundary Conditions:** Implement Neumann and Cauchy enforcement.
- **Element & Mesh Expansions:** Higher-order Nédélec elements, new geometries (prisms, hexahedra), and mesh formats.
- **Deeper HOFEM Integration:**
  - Matrix injection test.
  - Shared mesh structures.
- **Native Testing:** Use Julia to interactively test HOFEM.

University Degree in Data Science and Engineering  
2022-2026

# Parallel Programming of FEM in Julia

*Bachelor Thesis*

**Sergio Madrid Cuevas**

Tutors:

Sergio Llorente Romano

Luis Emilio García Castillo

Madrid, September 29, 2026

